

ASTERIA COMPOSITES LAB

Rapport d'architecture, d'implémentation et d'avancement — premier jet

Version 0.1.0	Date 2026-07-26	Branche codex/phase-0-1-first-draft
------------------	--------------------	--

Périmètre : architecture Phase 0 et jumeau numérique minimal Phase 1, exécutables de bout en bout avec données exclusivement synthétiques.

0. Répartition du travail

Le travail a été départagé par frontières de fichiers et responsabilités techniques afin que les contributions puissent progresser en parallèle sans mélanger les contrats, le moteur et la documentation.

Responsable	Périmètre	Livrable principal
Agent architecture	Décisions structurantes, documentation bilingue et six diagrammes Mermaid.	Architecture, méthodologie scientifique, hypothèses, contrats et budgets.
Agent contrats	Modèles Pydantic, chargeurs, configuration usine, exemples et schémas.	19 contrats publics, 19 schémas JSON et tests de validation.
Agent jumeau	Graphe, simulateur, KPI, exports, figures, CLI et tests fonctionnels.	Chaîne configuration vers simulation, événements, KPI et graphiques.
Agent principal	Cadrage, intégration, corrections, reproductibilité, CI, Git, tests et rapport.	Monorepo cohérent, validation globale, commits, Pull Request et PDF.

1. Résumé exécutif

Le premier jet livre un monorepo Python 3.12 exécutable de bout en bout. Il charge et valide une usine composite synthétique, construit un graphe orienté avec série, parallélisme, branche qualité et boucle de reprise, simule dix panneaux avec une graine fixe, exporte un journal et dix KPI, puis génère trois figures et un rapport Markdown. Les quatre autres modules métier sont cadrés par des contrats et une roadmap, mais ne sont pas encore implémentés.

2. Décisions structurantes

Monorepo modulaire au lancement

Un environnement, une CI et un verrou de dépendances réduisent le coût de la première livraison. Les packages et contrats conservent des frontières permettant une extraction ultérieure.

Contrats avant couplage

Les modules métier n'importent pas leurs composants internes respectifs. Les échanges passent par Pydantic, YAML/JSON et des schémas versionnés.

Simulation événementielle légère et lisible

Le premier moteur privilégie une baseline Python déterministe et testable. SimPy reste prévu derrière la même interface pour les calendriers, lots et ressources plus riches.

Données explicitement synthétiques

Les durées, coûts, consommations, défauts et pannes sont des hypothèses d'ingénierie. Aucun résultat n'est présenté comme une mesure industrielle.

Baselines avant méthodes avancées

Maintenance prédictive, CP-SAT, marketing bayésien et portefeuille R&D ne seront ajoutés qu'après stabilisation des invariants et comparés à une méthode simple.

3. Livrables réalisés

Bloc	État	Contenu
Architecture bilingue	Terminé	Cinq projets métier, package contracts, dépendances autorisées, risques, décisions reportées et six Mermaid.
Contrats v1.0.0	Terminé	19 contrats publics Pydantic, chargeurs YAML/JSON, validateurs et schémas JSON générés.
Usine de référence	Terminé	Deux produits, préparation, découpe, deux drapages parallèles, buffer limité, autoclave, finition, QC et reprise.
Simulation minimale	Terminé	Durées stochastiques avec graine, changements de série, dégradation, panne, maintenance, qualité, énergie et coûts.
KPI et exports	Terminé	Journal CSV, historique lots, résumé JSON, dix KPI, trois PNG et rapport Markdown.
Documentation produit	Terminé	README anglais/français à structure et liens identiques, catalogue des modules et roadmap détaillée.
Outils	Terminé	pyproject, uv.lock, Ruff, mypy strict, pytest, pre-commit et GitHub Actions.

4. Validation et qualité

Contrôle	Résultat	Observation
Installation propre	PASS	uv sync --frozen --extra dev dans un environnement neuf ; 45 packages installés.
Tests	PASS	17 tests pytest réussis en moins de 10 secondes.
Lint et format	PASS	Ruff check et Ruff format --check sans erreur.
Typage statique	PASS	mypy strict : 11 fichiers source, aucune erreur.
Topologie usine	PASS	10 nœuds, 10 arêtes, boucle QC/reprise, capacité layup égale à 2.
Déterminisme et invariants	PASS	Deux exécutions avec seed 42 produisent les mêmes événements et lots ; capacités respectées.
Budget fast	PASS	5,45 s de bout en bout avec trois figures ; cœur de simulation environ 0,002 s.
Parité bilingue	PASS	Structure Markdown des README et des six paires documentaires vérifiée automatiquement.

5. Avancement du programme

Phase	Avancement	Commentaire
Phase 0 — architecture et contrats	100 %	Documentation, contrats, configurations, schémas, outillage et tests livrés.
Phase 1 — jumeau numérique minimal	85 %	Chemin complet livré ; calendriers actifs, lots autoclave et stocks matière restent simplifiés.
Phase 2 — instrumentation et KPI	75 %	Événements, dix KPI, exports, figures et benchmark fast présents ; Gantt et conservation matière à enrichir.

Phase	Avancement	Commentaire
Phase 3 — dégradation et maintenance	20 %	Dégradation, pannes et réparation synthétiques dans le jumeau ; modèle prédictif non commencé.
Phases 4 à 6 — allocation, marketing, R&D	5 %	Contrats, interfaces et critères documentés ; algorithmes métier non implémentés.
Phases 7 et 8 — intégration et avancé	0 %	Scénarios intégrés et méthodes avancées volontairement reportés.

6. Traçabilité Git

Dépôt : Mariusppgn/Digital-Tween-4.0-Industry-

Branche : codex/phase-0-1-first-draft

Commits du premier jet : a33f35e, 3b88929, f0b8875

Pull Request : Pull Request brouillon créée après le commit de ce rapport.

7. Limites connues

- Le moteur événementiel est une baseline légère ; la migration vers SimPy doit préserver les interfaces et tests.
- Les calendriers opérateurs et technicien sont déclarés mais pas encore appliqués aux heures de début.
- L'autoclave est une ressource unitaire ; la constitution et la compatibilité détaillées des lots ne sont pas modélisées.
- Le stock intermédiaire est borné dans l'ordonnancement, mais les inventaires matière détaillés ne sont pas encore conservés.
- Les lois de durée, défaut, panne, énergie et coût sont synthétiques et non calibrées sur une usine réelle.
- Maintenance prédictive, allocation, marketing et R&D sont documentés et contractés, mais non exécutables à ce stade.
- Le benchmark disponible concerne une seule machine Windows 11 ; une campagne multiplateforme reste nécessaire.

8. Prochaines étapes recommandées

1. Appliquer les calendriers d'équipes, compétences opérateurs et disponibilité du technicien dans le moteur.
2. Modéliser les lots autoclave, la compatibilité des produits et la conservation explicite des stocks matière.
3. Ajouter les données capteurs synthétiques et comparer corrective, préventive, seuils, EWMA et Weibull.
4. Construire la baseline gloutonne puis le modèle CP-SAT d'allocation des ressources.
5. Publier des figures de référence, un Gantt et un benchmark fast/standard multiplateforme.
6. Créer les README bilingues indépendants de chaque module lors de leur implémentation.

Conclusion — Le premier jet fournit un socle logiciel cohérent et vérifiable. Il démontre l'intégration complète configuration □ validation □ simulation □ événements □ KPI □ exports □ figures, tout en maintenant une séparation nette entre données synthétiques, hypothèses d'ingénierie et futures méthodes scientifiques.